

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I **SWETA R (192511408)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SWETA R

Assessment Tasks
Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

ANSWERS:

1. Uneven Input Splits and Idle Reducers

Error:

The likely design error is **poor input partitioning**, causing some mapper tasks to receive much more data than others.

Analysis:

Large input splits take longer to process, while other reducers remain idle waiting for intermediate data.

Corrective Actions:

- Use suitable and consistent **input split/block sizes**.
- Balance the input data across mapper tasks.
- Use an appropriate **partitioner** to distribute intermediate data evenly.
- Optimize the number of reducers according to the workload.

Result:

Balanced task distribution reduces processing time and improves cluster utilization.

2. HDFS Data Unavailability After Node Failure

Error:

The probable error is **insufficient replication** or improper **NameNode High Availability** configuration.

Analysis:

If data has only one copy, failure of its DataNode makes the data unavailable. A single NameNode can also become a single point of failure.

Corrective Actions:

- Maintain an appropriate HDFS **replication factor**, commonly 3.
- Distribute replicas across different DataNodes/racks.
- Configure **Active and Standby NameNodes**.
- Monitor DataNode health and automatically re-replicate missing blocks.

Result:

Replication and NameNode HA improve data availability and fault tolerance.

3. Duplicate Records in ETL and Apache NiFi

Error:

The likely causes are **repeated data ingestion, incorrect ETL logic, or lack of duplicate detection**.

Analysis:

NiFi may process the same file or message more than once, while the analytics store may accept repeated records without checking unique identifiers.

Corrective Actions:

- Use unique **record IDs** for duplicate detection.
- Configure NiFi flow controls to prevent repeated processing.
- Apply **deduplication and validation** during ETL.
- Use idempotent database operations such as **upsert** where appropriate.

Result:

These controls ensure each logical record is stored only once.

4. YARN Resource Starvation

Error:

The scheduling error is likely **improper resource allocation** or allowing one user's jobs to consume most cluster resources.

Analysis:

When multiple users submit jobs simultaneously, large jobs may occupy most containers, leaving insufficient resources for other users.

Corrective Actions:

- Use **Capacity Scheduler or Fair Scheduler**.
- Create separate queues for different users or workloads.
- Set resource limits and minimum resource guarantees.
- Assign priorities to important jobs.

Result:

Proper scheduling ensures fair resource sharing and prevents resource starvation.

5. Outdated Data After Backup Recovery

Error:

The probable design flaw is the use of **outdated backups or improperly synchronized replicas**.

Analysis:

If backups are not updated regularly, recovery may restore an older version of the data. Replication alone may also not protect against logical errors or accidental deletion.

Corrective Actions:

- Schedule **regular incremental and full backups**.
- Maintain multiple backup versions.
- Monitor replication and backup status.
- Use timestamps/versioning to identify the latest valid backup.
- Periodically **test recovery procedures**.

Result:

Versioned and regularly verified backups ensure the latest valid data can be restored after failure.